PRO2000 - PROGRAMMING

Week 43- Inheritance, polymorphism

Gebremariam Assres
Gebremariam.assres@kristiania.no

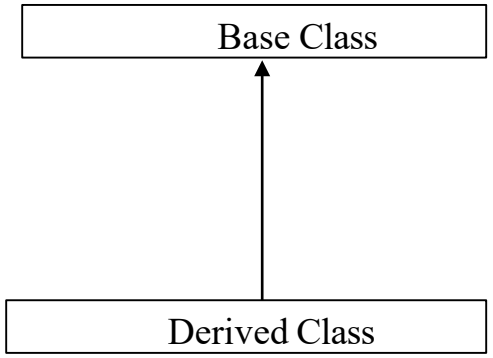
Inheritance

- Inheritance is a way of creating a new class by starting with an existing class and adding new members
- The new class can replace or extend the functionality of the existing class
- Inheritance models the 'is-a' relationship between classes
- The existing class is called the base class, parent class or superclass
- The new class is called the derived class, child class or subclass

Inheritance ...

```
// Existing class
class Base
{
};
// Derived class
class Derived : public Base
{
};
```

Inheritance Class Diagram




```
class Parent
{
    int a;
    void bf();
};
class Child: public Parent
{
    int c;
    void df();
};
```

- Objects of Parent have members: `int a; void bf();`
- Objects of Child have members: `int a; void bf(); int c; void df();`

Inheritance ...

- Protected member access specification:
  - A class member labeled protected is accessible to member functions of derived classes and member functions of same class
  - It's like private, except accessible to members functions of derived classes
- Base class access specification
  - determines how private, protected, and public members of base class can be accessed by derived classes

Inheritance ...

- C++ supports three inheritance or base class access modes:

```
class Child : public Parent { };
class Child : protected Parent{ };
class Child : private Parent{ };
```

- Base class access is not the same as member access specification:
  - Base class access determines access for inherited members
  - Member access specification determine access for members defined in the class

Inheritance ...

- public – object of derived class can be treated as object of base class (not vice-versa)
- protected– more restrictive than public, but allows derived classes to know some of the details of parents
- private– prevents objects of derived class from being treated as objects of base class.

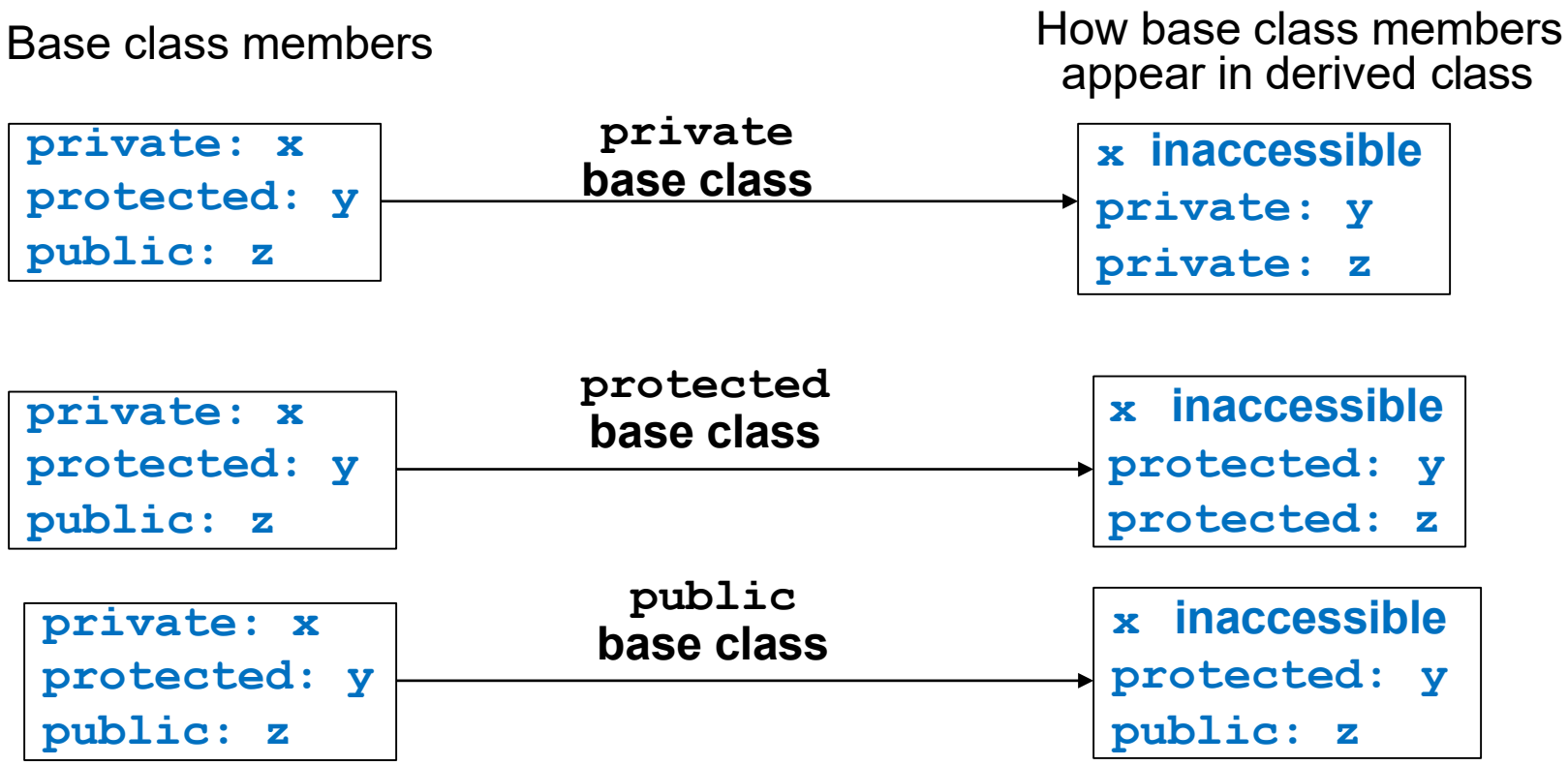
```
class Child : public Parent
{
   protected: int a;
   public:
        Child();

};
```

base access

member access

Inheritance ...



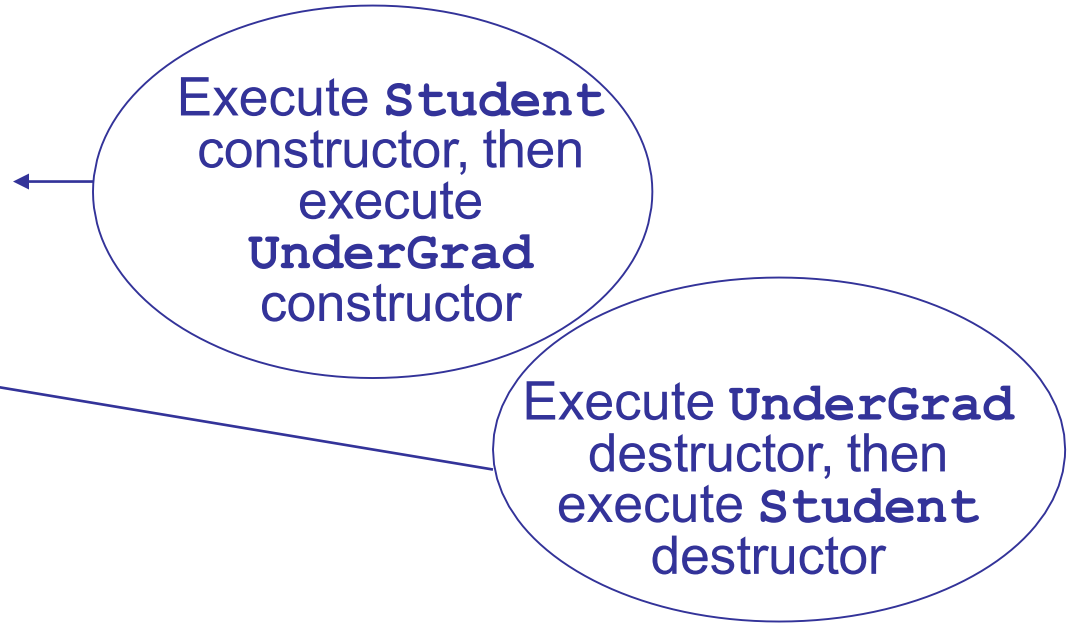
Inheritance ...

- Constructors, destructors and inheritance:
  - By inheriting every member of the base class, a derived class object contains a base class object.
  - The derived class constructor can specify which base class constructor should be used to initialize the base class object.
  - When object of derived class is created, constructor of base class is executed first followed by derived class's constructor.
  - When an object of a derived class is destroyed, its destructor is called first, then that of the base class.

Inheritance ...

- Example of order of execution

```
// Student - base class
// UnderGrad - derived class
// Both have constructors, destructors
int main()
{
    UnderGrad u1;
    ...
    return 0;
}// end main
```



Inheritance ...

- Passing arguments to base class constructor
  - Allows selection between multiple base class constructors
  - Specify arguments to base constructor on derived constructor heading
  - Can also be done with inline constructors
  - Must be done if base class has no default constructor

```
class Parent {
      int x, y;
      public: Parent(int,int);
};
class Child : public Parent {
      int z
      public:
        Child(int a): Parent(a,a*a)
        {z = a;}
};
```

Inheritance ...

- Overriding base class functions
  - If a function in a derived class that has same name and parameter list as a function in the base class, it is called overriding.
  - It is used to replace a function in base class with different actions in derived class.
  - When a function is overridden, all objects of derived class use the overridden function.
  - If necessary to access the overridden function, it can be done using base class name and scope resolution operator: Student::getName();

Polymorphism

- Polymorphism is an object-oriented feature that allows a function, operator or object to behave differently with context.
- Function overloading- multiple functions share the same name but have different parameters (types or numbers).
- Operator overloading- redefine behavior of existing operators to work seamlessly with user-defined types (classes).

Polymorphism ...

- This pointer:
  - this pointer is an implicit parameter passed to a member function and points to the object calling the function
  - Can be used to access members that may be hidden by parameters with the same name:

```
class SomeClass
{
    private:
        int num;
    public:
        void setNum(int num)
        { this->num = num; }
};
```

Polymorphism ...

- Constant member function does not modify its calling object:
  - When `const` appears in the parameter list, the function is prevented from modifying the parameter: int setNum (const int num)
  - When `const` follows the parameter list, the function is prevented from modifying the object: int getX() const
- Static member variable:
  - one instance of variable for the entire class and shared by all objects of the class.
- Static member function:
  - can be used to access static member variables and called before any class objects are created.

Polymorphism ...

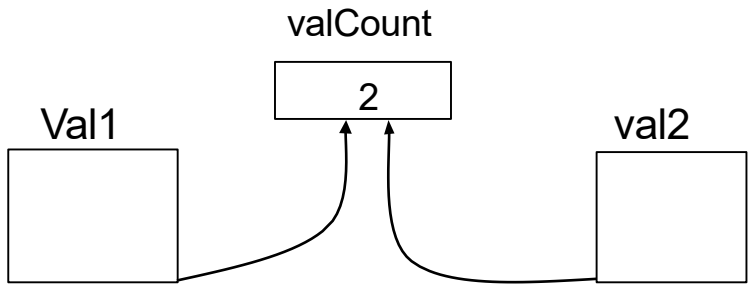
- Static member variables
  - Must be declared in class with keyword static:

```
class IntVal {
public:
        intVal(int val = 0)
        { value = val; valCount++ }
        int getVal();
        void setVal(int);
private:
        int value;
        static int valCount;
};
```

  - Must be defined outside of the class:

```
class IntVal {
        …
};
//Definition outside of class
int IntVal::valCount = 0;
```

  - Accessed or modified by any object of the class and modifications by one object are visible to all objects of the class: IntVal val1, val2;



Polymorphism ...

- Static member functions

1) Declared with static before return type:

```
class IntVal {
public:
        static int getValCount()
        { return valCount; }
private:
        int value;
        static int valCount;
};
```

2) Can be called independently of class objects, through the class name: `cout << IntVal::getValCount();`
3) Because of item 2 above, the this pointer cannot be used.
4) Can be called before any objects of the class have been created.
5) Used primarily to manipulate static member variables of the class.

Polymorphism ...

- A friend is a function or class that is not a member of another class but can access its private and protected members.

Friend function:

```
class A {
private: int x;
   friend void setValue(A&, int);
};
void setValue(A& object, int value) {
   object.x = value;
}
```

Friend member function:

```
class A {
private: int x;
   friend void B::setValue(A&, int);
};
```

Friend class:

```
class A {
private: int x;
   friend class B;
};
class B {
public:
   void setValue(A& object, int value) {
      object.x = value;
   }
   int getValue(A& object) {
      return object.x;
   }
};
```

Polymorphism ...

- Copy constructor:
  - Memberwise assignment copies all data members from one object to another using =: V v1, v2; v2 = v1; (assignment) V v3 = v2; (initialization)
  - A copy constructor is special constructor used when a newly created object is initialized to the data of another object of same class.
  - A default copy constructor copies field-to-field using memberwise assignment.
  - Problems occur when objects contain pointers variables.

Polymorphism ...

- A copy constructor is called when
  - An object is initialized from an object of the same class
  - An object is passed by value to a function
  - An object is returned using a **`return`** statement from a function

```
class CpClass{
int *p;
public:
    CpClass(const CpClass &obj)
    { p = new int; *p = *obj.p; }
    CpClass(int v=0)
    { p = new int; *p = v; }
    ~CpClass(){delete p;}
};
```

Polymorphism ...

- Operator overloading
  - Operators such as **=**, **+**, and others can be redefined for use with objects of a class
  - The name of the function for the overloaded operator is operator followed by the operator symbol
    - *e.g.*, operator+ is the overloaded **+** operator and operator= is the overloaded **=** operator
  - Operators can be overloaded as instance member functions, or as friend functions.
  - The overloaded operator must have the same number of parameters as the standard version.

Polymorphism ...

- Operator overloading …
  - An overloaded binary operator as instance member needs only one parameter which represents operand on the right, and left operand is the calling object.

```
class OpClass{
    private: int x;
    public:
        OpClass operator+(OpClass right);
};
```

  - The implicit left parameter is accessed through this pointer:

```
OpClass OpClass::operator+(OpClass r){
    OpClass sum;
    sum.x = this->x + r.x; return sum;
}
```

  - Operator can be invoked as a member function: OpClass a, b, s; s = a.operator+(b);
  - It can also be invoked in the more conventional manner: OpClass a, b, s; s = a + b;

Polymorphism ...

- Overloading types of operators
  - Overloading the assignment operator solves problems with object assignment when an object contains a pointer.
  - ++, --operators overloaded differently for prefix vs. postfix notation
  - Overloaded relational operators should return a boolean value

  - Overloaded stream operators >>, << must return istream, ostream objects and take istream, ostream objects as parameters
  - Overloaded [ ] operator can be used to create classes that behave like arrays, providing bounds-checking on subscripts

Polymorphism ...

- Function overloading:
  - Allows multiple functions to share the same name and scope, provided their arguments differ in type, number, or sequence.
  - The compiler resolves which function to call prior to runtime based on the arguments passed, ensuring zero execution overhead.
  - Cannot be differentiated by return type alone; altering only the return type results in a compilation error due to call ambiguity.
  - Example:

```
// Different number of parameters
void calculateArea(double radius);
void calculateArea(double length, double width);

// Different parameter types
void printData(int integerVal);
void printData(const string& stringVal);
```

Polymorphism ...

- Templates:
  - Templates provide compile-time polymorphism.
  - They allow the same function or class to work with different data types without inheritance or virtual functions.
    - Write one class or function that works with many data types.
    - Errors are detected during compilation.
    - avoid run-time overhead as code is generated at compile time.

Polymorphism ...

- In the following code, the compiler creates the appropriate version of printValue() for each type.
  - This is called compile-time polymorphism, unlike virtual functions, which provide run-time polymorphism.

```
#include <iostream>
using namespace std;

template <typename T>
void printValue(T value) {
   cout << value << endl;
}
```

```
int main() {
   printValue(10);        // int
   printValue(3.14);      // double
   printValue("Hello");   // string literal

   return 0;
}
```

Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on class inheritance, and polymorphism.*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)